Secure architecture - Firewalls

Outlines

1 Introduction
- Presentation
- Firewall classification
- Limits and vulnerabilities

Firewall

Security ⇒ limit connections

- to the outside (private information)
- to the inside (malware)

How to?

⇒ separate "right" bits from the "wrong" ones

- **Firewalls** control network access with clearance/refusal according to specific criteria.

→ Supports every security level (from relaxed to very strict)

Firewall

Security ⇒ limit connections

- to the outside (private information)
- to the inside (malware)

How to?

⇒ separate "right" bits from the "wrong" ones

- **Firewalls** control network access with clearance/refusal according to specific criteria.

→ Supports every security level (from relaxed to very strict)

Hardware firewall

- Quality
- Robustness
- Firewall functionalities can be distributed
  - Intranets, extranets
  - VLAN, VPN
  - ⇒ Connexion control and security

Bastions

- Specialized or not, located as first line of defence
- Rely on the OS ⇒ it must be secure!
- Its capacity depends on used services → be adaptive
- Located in a physical secure place
- Distribute services over several bastions as much as possible

Bastions

- Specialized or not, located as first line of defence
- Rely on the OS ⇒ it must be secure!
- Its capacity depends on used services → be adaptive
- Located in a physical secure place
- Distribute services over several bastions as much as possible

Bastions

- Specialized or not, located as first line of defence
- Rely on the OS ⇒ it must be secure!
- Its capacity depends on used services → be adaptive
- Located in a physical secure place
- Distribute services over several bastions as much as possible

Bastions

- Specialized or not, located as first line of defence
- Rely on the OS ⇒ it must be secure!
- Its capacity depends on used services → be adaptive
- Located in a physical secure place
- Distribute services over several bastions as much as possible

Bastions

- Specialized or not, located as first line of defence
- Rely on the OS $\Rightarrow$ it must be secure!
- Its capacity depends on used services $\rightarrow$ be adaptive
- Located in a physical secure place
- Distribute services over several bastions as much as possible

Network firewalls

- Objective = **filter** IP packets
- **Rules** define the decision to take with regards to information inside packets
- Packet analyse based on
  - header contents (fragmentation?)
  - *Source* and *Destination* @ (*smurf*)
  - options (*source routing*...)

\+ Handle more data than "Application" firewalls

− When a service is already attacked, the firewall is useless.

- Every IP protocol can be scanned → what to do with ICMP?

⇒ Accept only specific packets (ICMP types 3 "`Destination unreachable`" and 4 "`Source quench`")

Network firewalls

- Objective = **filter** IP packets
- **Rules** define the decision to take with regards to information inside packets
- Packet analyse based on
  - header contents (fragmentation?)
  - *Source* and *Destination* @ (*smurf*)
  - options (*source routing*...)

\+ Handle more data than "Application" firewalls

− When a service is already attacked, the firewall is useless.

- Every IP protocol can be scanned → what to do with ICMP?

⇒ Accept only specific packets (ICMP types 3 "`Destination unreachable`" and 4 "`Source quench`")

Network firewalls

- Objective = **filter** IP packets
- **Rules** define the decision to take with regards to information inside packets
- Packet analyse based on
  - header contents (fragmentation?)
  - *Source* and *Destination* @ (*smurf*)
  - options (*source routing*...)

\+ Handle more data than "Application" firewalls

− When a service is already attacked, the firewall is useless.

- Every IP protocol can be scanned → what to do with ICMP?

⇒ Accept only specific packets (ICMP types 3 "`Destination unreachable`" and 4 "`Source quench`")

Network firewalls

- Objective = **filter** IP packets
- **Rules** define the decision to take with regards to information inside packets
- Packet analyse based on
  - header contents (fragmentation?)
  - *Source* and *Destination* @ (*smurf*)
  - options (*source routing*...)

\+ Handle more data than "Application" firewalls

− When a service is already attacked, the firewall is useless.

- Every IP protocol can be scanned → what to do with ICMP?

⇒ Accept only specific packets (ICMP types 3 "`Destination unreachable`" and 4 "`Source quench`")

Network firewalls

- Objective = **filter** IP packets
- **Rules** define the decision to take with regards to information inside packets
- Packet analyse based on
  - header contents (fragmentation?)
  - *Source* and *Destination* @ (*smurf*)
  - options (*source routing*...)

\+ Handle more data than "Application" firewalls

− When a service is already attacked, the firewall is useless.

- Every IP protocol can be scanned → what to do with ICMP?

⇒ Accept only specific packets (ICMP types 3 "`Destination unreachable`" and 4 "`Source quench`")

Network firewalls

- Objective = **filter** IP packets
- **Rules** define the decision to take with regards to information inside packets
- Packet analyse based on
  - header contents (fragmentation?)
  - *Source* and *Destination* @ (*smurf*)
  - options (*source routing*...)

\+ Handle more data than "Application" firewalls

− When a service is already attacked, the firewall is useless.

- Every IP protocol can be scanned → what to do with ICMP?

⇒ Accept only specific packets (ICMP types 3 "`Destination unreachable`" and 4 "`Source quench`")

Transport firewalls

- Complement Network firewalls
- Scan *Source* and *Destination* **ports**
- Scan flags SYN, ACK...

→ stateful firewalls detect connexion requests and separate client requests (SYN) from server responses.

⇒ Example: packets with ACK = "on" are refused when there is no connexion.

Transport firewalls

- Complement Network firewalls
- Scan *Source* and *Destination* **ports**
- Scan flags SYN, ACK...

→ stateful firewalls detect connexion requests and separate client requests (SYN) from server responses.

⇒ Example: packets with ACK = "on" are refused when there is no connexion.

Transport firewalls

- Complement Network firewalls
- Scan *Source* and *Destination* **ports**
- Scan flags SYN, ACK...

→ stateful firewalls detect connexion requests and separate client requests (SYN) from server responses.

⇒ Example: packets with ACK = "on" are refused when there is no connexion.

Application firewalls

- Mask local @
- Use **proxies** which fit specific protocol with specific filtering rules

→ Go-between between client and server for entering and exiting connexions

- Ideal proxy: rewrite headers to minimize information
- Hacker requests have got a specific form

→ Limit headers, character sets...

Application firewalls

- Mask local @
- Use **proxies** which fit specific protocol with specific filtering rules

→ Go-between between client and server for entering and exiting connexions

- Ideal proxy: rewrite headers to minimize information
- Hacker requests have got a specific form

→ Limit headers, character sets...

Application firewalls

- Mask local @
- Use **proxies** which fit specific protocol with specific filtering rules

→ Go-between between client and server for entering and exiting connexions

- Ideal proxy: rewrite headers to minimize information
- Hacker requests have got a specific form

→ Limit headers, character sets...

Application firewalls

- Mask local @
- Use **proxies** which fit specific protocol with specific filtering rules

→ Go-between between client and server for entering and exiting connexions

- Ideal proxy: rewrite headers to minimize information
- Hacker requests have got a specific form

→ Limit headers, character sets...

Application firewalls

- Mask local @
- Use **proxies** which fit specific protocol with specific filtering rules

→ Go-between between client and server for entering and exiting connexions

- Ideal proxy: rewrite headers to minimize information
- Hacker requests have got a specific form

→ Limit headers, character sets...

Hybrid firewalls

How to consider "exotic" protocols?

- *Generic Service Proxy*

⇒ Combine proxy and packet filter

- Best idea: distinct subnets

\+ distinct services → localized attacks

− High costs

Hybrid firewalls

How to consider "exotic" protocols?

- *Generic Service Proxy*

⇒ Combine proxy and packet filter

- Best idea: distinct subnets

\+ distinct services → localized attacks

− High costs

Firewall limits

- Necessary update to follow network changes
- Stopping exit packets can be hard according to services and protocols (UDP more difficult to handle than TCP)
- Very difficult to retrieve packet information only from a fragment

→ Never protects again virus

- Always some software flaws
- Some attacks can not be avoided (Denial Of Service)

Firewall limits

- Necessary update to follow network changes
- Stopping exit packets can be hard according to services and protocols (UDP more difficult to handle than TCP)
- Very difficult to retrieve packet information only from a fragment

→ Never protects again virus

- Always some software flaws
- Some attacks can not be avoided (Denial Of Service)

Firewall limits

- Necessary update to follow network changes
- Stopping exit packets can be hard according to services and protocols (UDP more difficult to handle than TCP)
- Very difficult to retrieve packet information only from a fragment

→ Never protects again virus

- Always some software flaws
- Some attacks can not be avoided (Denial Of Service)

Firewall limits

- Necessary update to follow network changes
- Stopping exit packets can be hard according to services and protocols (UDP more difficult to handle than TCP)
- Very difficult to retrieve packet information only from a fragment

→ Never protects again virus

- Always some software flaws
- Some attacks can not be avoided (Denial Of Service)

Proxy limits

- Only classical protocols are identified
- Imagining every attack that could affect a protocol is not easy

→ Some attacks can reach their target whereas legitimate connexions can be refused

⇒ "Sticking" to the protocol is not always enough... (command `CONNECT` de HTTP 1.0 → open connection to any remote port number)

- Cruder firewalls are less error-prone
- Security enhanced → performance bottleneck

Proxy limits

- Only classical protocols are identified
- Imagining every attack that could affect a protocol is not easy

→ Some attacks can reach their target whereas legitimate connexions can be refused

⇒ "Sticking" to the protocol is not always enough... (command `CONNECT` de HTTP 1.0 → open connection to any remote port number)

- Cruder firewalls are less error-prone
- Security enhanced → performance bottleneck

Encapsulating attacks

A vulnerable application is called by another one associated with a specific protocol.

→ Ex.: CGI scripts called by HTTP server

→ Ex.: mail managers handled by a mail server

⇒ A script without parameter filtering can execute arbitrary commands using specific characters

⇒ Nearly impossible to filter every character...

Encapsulating attacks

A vulnerable application is called by another one associated with a specific protocol.

→ Ex.: CGI scripts called by HTTP server

→ Ex.: mail managers handled by a mail server

⇒ A script without parameter filtering can execute arbitrary commands using specific characters

⇒ Nearly impossible to filter every character...

Attack with non filtering transmission

Use a protocol with binay data (SMTP, IMAP...)

- The firewall can not filter those data

→ Security relies on the application using this protocol.

- A solution: delete every suspect-looking packet?

– Too much possibilities to check

– Ressource cost would be prohibitive

⇒ Use an **Intrusion Detection System**

Attack with non filtering transmission

Use a protocol with binay data (SMTP, IMAP...)

- The firewall can not filter those data

→ Security relies on the application using this protocol.

- A solution: delete every suspect-looking packet?

– Too much possibilities to check

– Ressource cost would be prohibitive

⇒ Use an **Intrusion Detection System**

Attack with non filtering transmission

Use a protocol with binay data (SMTP, IMAP...)

- The firewall can not filter those data

→ Security relies on the application using this protocol.

- A solution: delete every suspect-looking packet?

– Too much possibilities to check

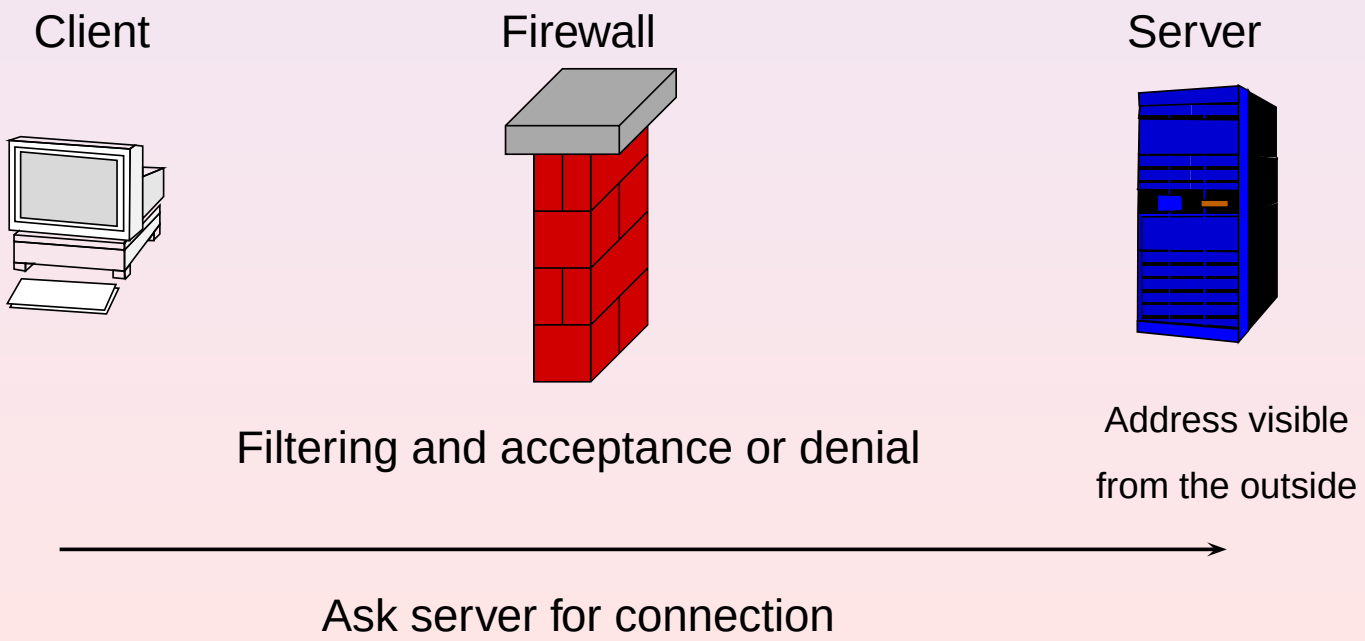
– Ressource cost would be prohibitive

⇒ Use an **Intrusion Detection System**

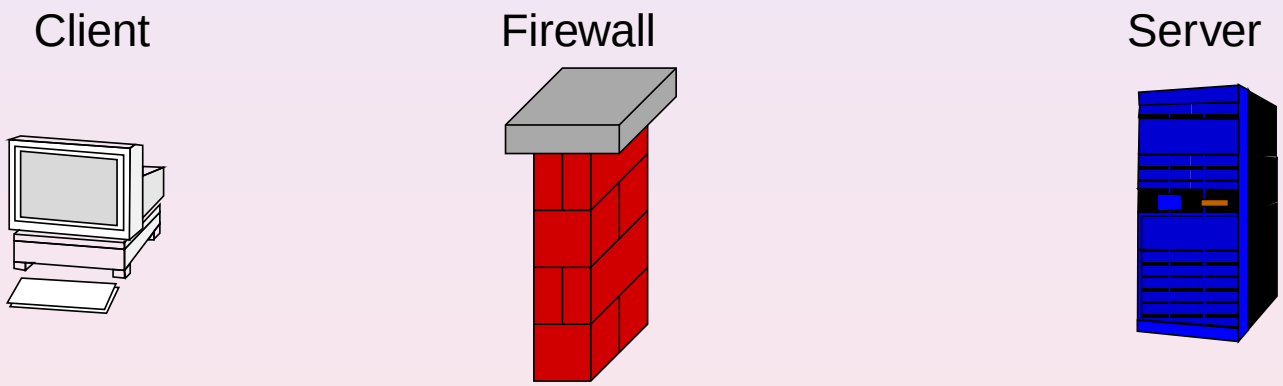
Figures

Network-level firewall

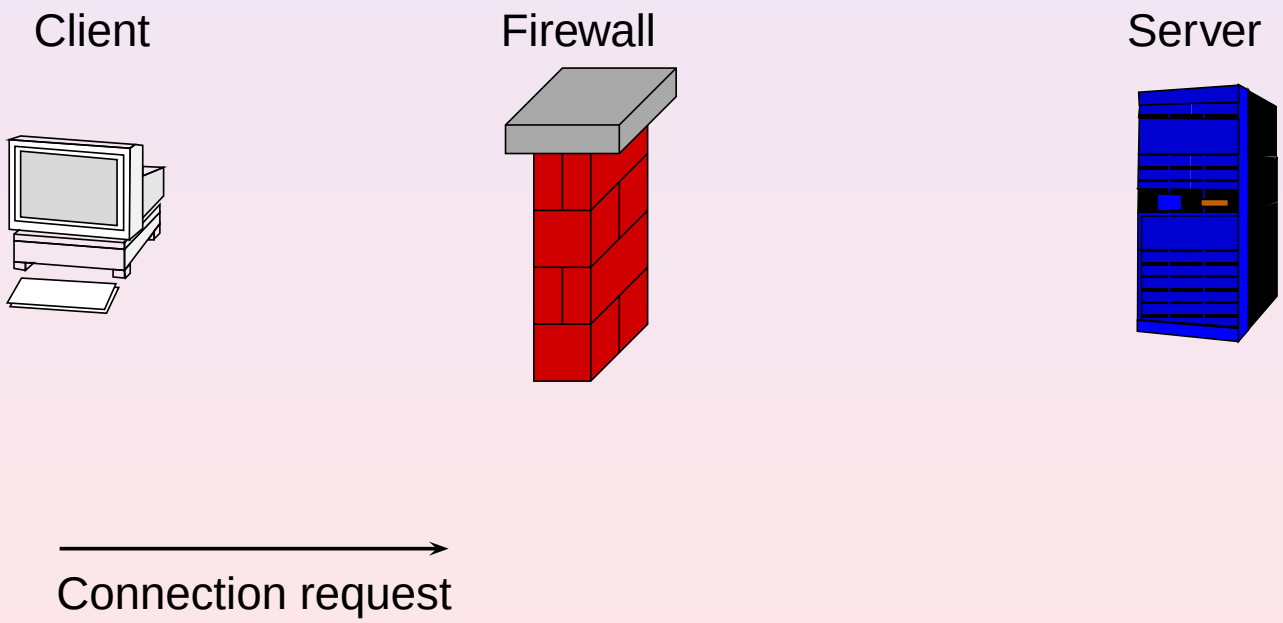
Back



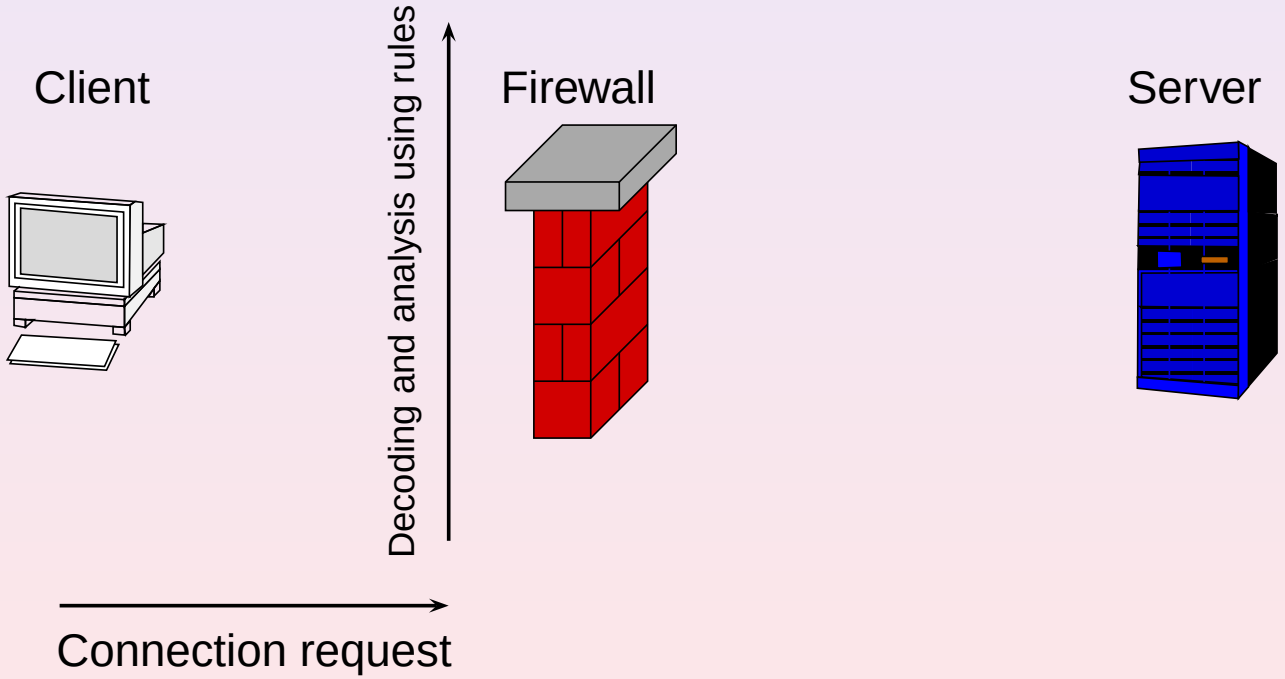
Application-level firewall 1/5



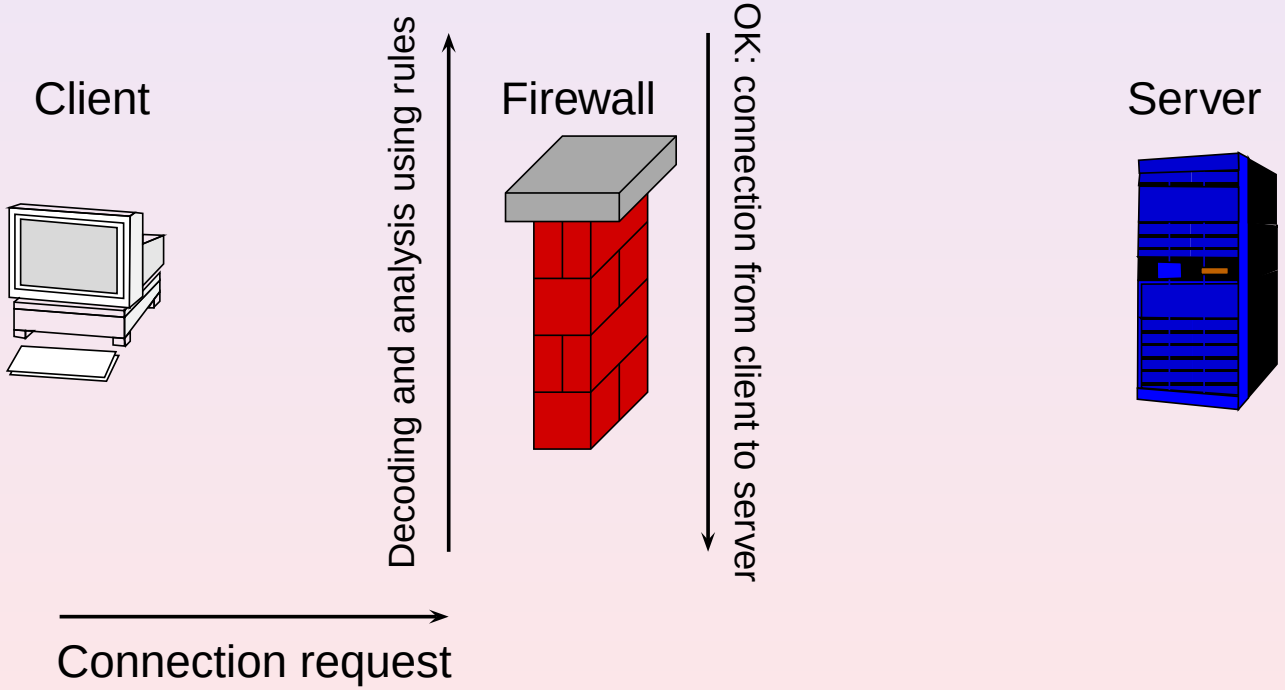
Application-level firewall 2/5



Application-level firewall 3/5

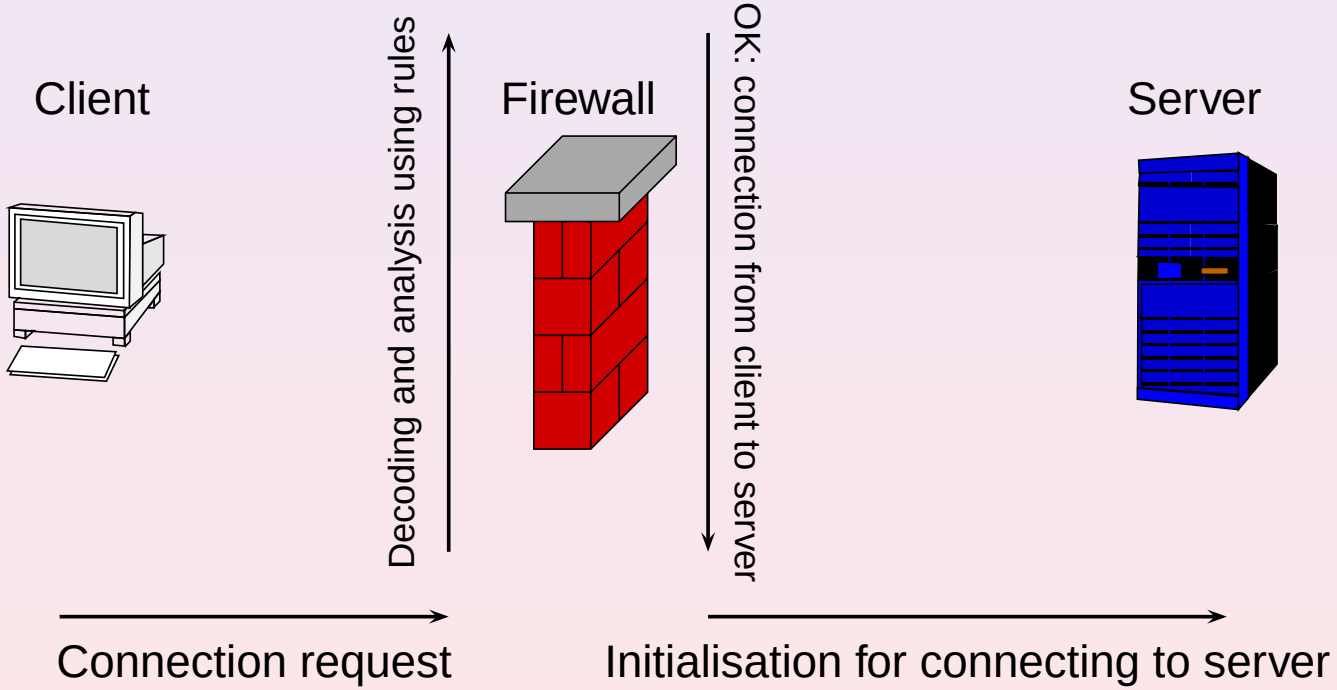


Application-level firewall 4/5



Application-level firewall 5/5

Back



Proxy test

Back

- Client :

```
GET chemin/page.html HTTP/1.1
Accept:  image/gif, image.jpeg, */*
Accept-language:fr
Accept-Encoding:gzip, deflate
User-Agent:Mozilla/4.0
Host:  Nom.du.serveur.org
```

- Server :

```
HTTP/1.1 200 OK
Date:  Mon,18 Oct 2004 01:02:03 GMT
Server:  Apache/1.3.9 (Unix) Mandrake PHP/3.0.18
X-Powered by:  PHP/3.0.18
Content-Type:  texte/html
<html><body> What a great page !  </html></body>
```